

Are You Getting What You Pay For?

Auditing Model Substitution in LLM APIs

Why Software Tests Fail and Hardware Attestation Wins

The Model Substitution Problem: Economic Incentives Drive Covert Swaps

Black-box APIs provide no guarantee the advertised model is actually being served



Smaller Model

e.g., 8B instead of 70B

HIGH RISK

Largest performance gap,
most detectable



Quantized Variant

FP8/INT8 instead of FP16

STEALTHY

Minimal benchmark impact,
nearly undetectable



Fine-tuned Version

Updated or retrained variant

SUBTLE

Same architecture,
shifted behavior



Different Family

Entirely different architecture

BLATANT

Most detectable,
but still possible

Formal Hypothesis Testing Framework

Can an auditor distinguish the specified model distribution from the actual one?

H0 (Null): $P_{\text{actual}}(y|x) = P_{\text{spec}}(y|x)$
The API serves the specified model

H1 (Alt): $P_{\text{actual}}(y|x) \neq P_{\text{spec}}(y|x)$
The API serves a substitute

Incentive: Cut GPU costs, keep pricing
Strong economic motivation for substitution

FP8 Quantization Preserves Most Accuracy

Making Substitution Hard to Detect

Key insight: FP8 scores fall within error bars of the original, especially on MMLU and GSM8K

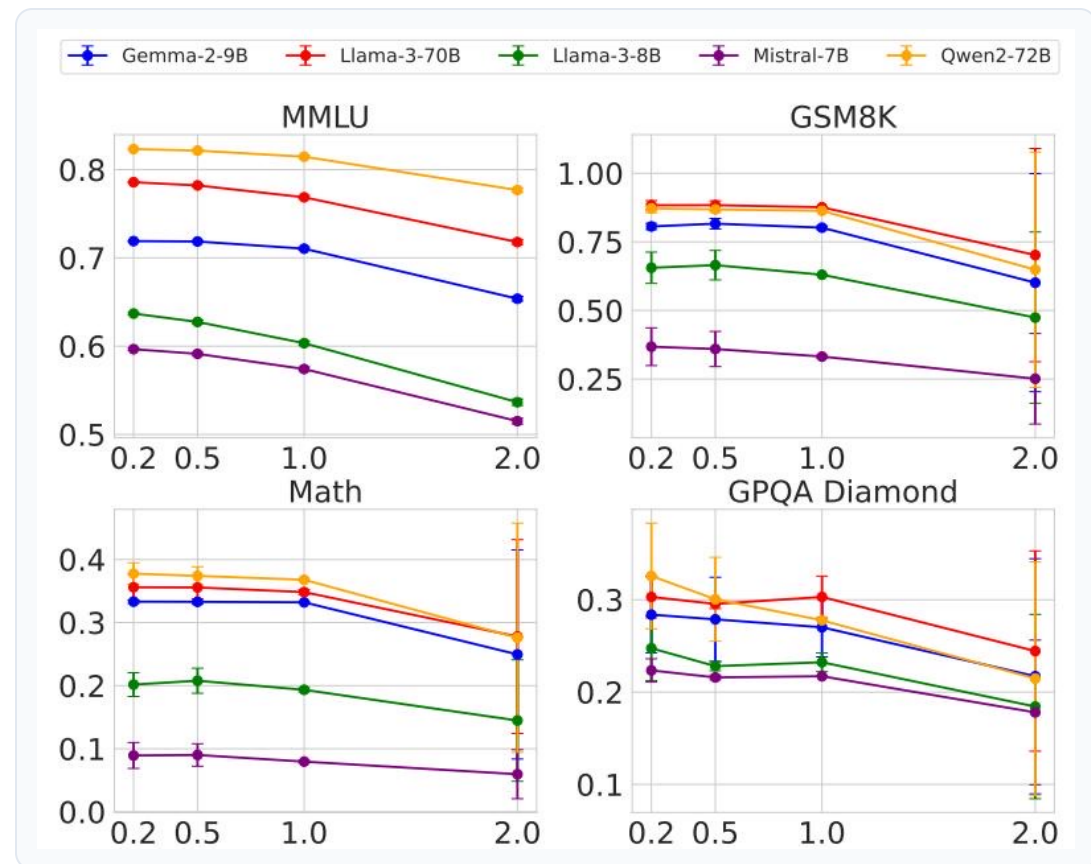
Model	MMLU	GSM8K	MATH	GPQA Diamond
Llama-3-8B-Instruct	62.69 ± 0.18	61.14 ± 3.47	20.65 ± 3.43	22.62 ± 0.22
Llama-3-8B-Instruct-FP8	62.43 ± 0.26	60.90 ± 4.10	14.91 ± 2.49	20.14 ± 0.24
Llama-3-70B-Instruct	78.05 ± 0.08	88.06 ± 1.44	35.69 ± 1.33	29.60 ± 0.51
Llama-3-70B-Instruct-FP8	77.88 ± 0.13	87.35 ± 1.24	35.75 ± 1.16	33.12 ± 0.30
Gemma-2-9b-it	71.86 ± 0.08	81.80 ± 1.35	33.41 ± 0.28	28.64 ± 2.97
Gemma-2-9b-it-FP8	71.92 ± 0.11	79.41 ± 1.14	32.53 ± 0.34	27.81 ± 3.22
Qwen2-72B-Instruct	82.18 ± 0.08	86.72 ± 1.00	37.39 ± 1.41	29.93 ± 2.71
Qwen2-72B-Instruct-FP8	81.98 ± 0.08	86.82 ± 0.97	37.67 ± 1.39	31.08 ± 1.96
Mistral-7B-Instruct-v0.3	59.15 ± 0.10	35.90 ± 4.54	8.94 ± 1.24	21.60 ± 0.17
Mistral-7B-Instruct-v0.3-FP8	58.77 ± 0.13	32.20 ± 4.02	7.68 ± 1.23	22.72 ± 0.19

FP8 within error bars FP8 outside error bars

Source: arxiv.org/abs/2504.04715

Higher Temperature Amplifies Performance Gaps

but Increases Noise — Complicating Statistical Detection



1

Temperature 2.0 degrades all models

Performance drops across all five model families at the highest temperature setting

2

Gap narrows or becomes noisy

The difference between original and substitute models shrinks at higher temperatures

3

Four benchmarks tested: T = 0.2, 0.5, 1.0, 2.0

MMLU, GSM8K, MATH, and GPQA Diamond all show similar temperature-dependent patterns

4

Statistical detection becomes unreliable

High temperature noise obscures the already subtle substitution signal

Implication: Temperature variation is not a reliable tool for distinguishing model substitution

Log Probability Fingerprinting Fails

Due to Inference Nondeterminism

Same model, same weights — different log probabilities across software and hardware

Model	BERT Acc	T5 Acc	GPT2 Acc	LLM2Vec Acc
Llama3-70B-Instruct-FP8	50.55	50.10	50.45	49.90
Llama3-70B-Instruct-INT8	51.60	49.90	51.30	50.25
Gemma2-9b-it-FP8	49.95	50.30	51.20	49.80
Gemma2-9b-it-INT8	49.00	49.70	51.65	49.55
Mistral-7b-v3-Instruct-FP8	50.55	49.75	48.70	48.75
Mistral-7b-v3-Instruct-INT8	49.70	50.75	50.50	51.15
Qwen2-72B-Instruct-FP8	50.05	50.25	48.75	49.80
Qwen2-72B-Instruct-INT8	50.75	50.55	49.45	50.20

Figure: Log probability variation across inference frameworks and hardware for Llama-3-8B



Software Framework Variation

transformers 4.40 vs 4.50

vLLM 0.6.2 vs 0.8.2

Different framework versions produce different log probabilities for the same model weights

Cannot distinguish version drift from substitution



Hardware Variation

NVIDIA A100 GPU

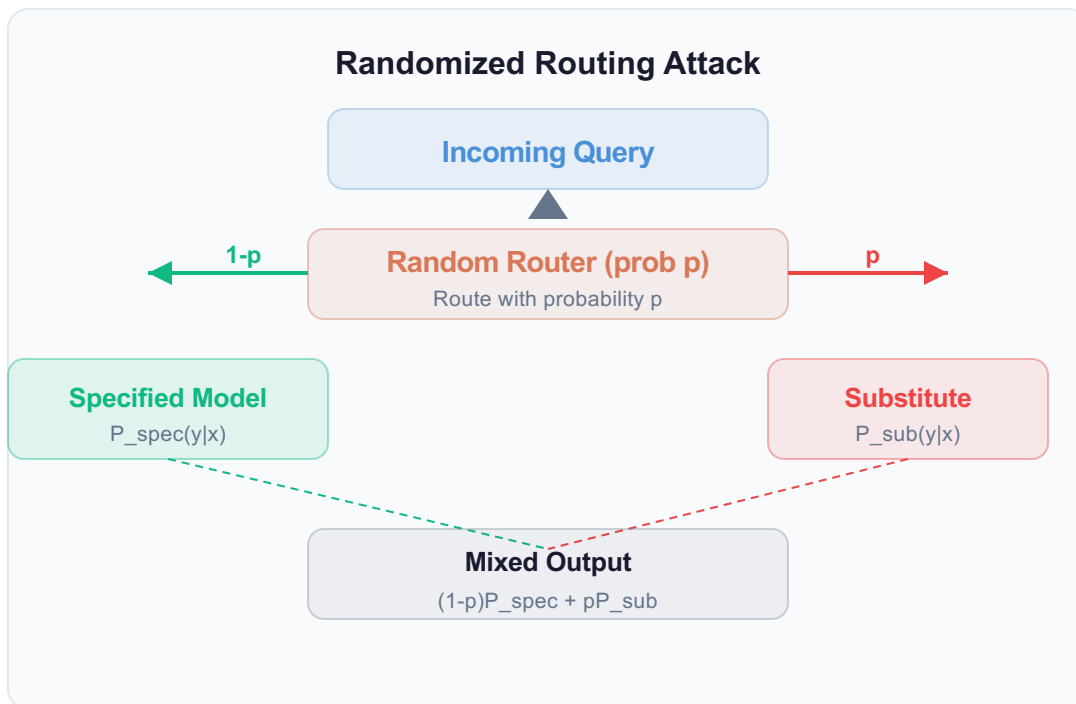
NVIDIA H100 GPU

Same model on different GPU hardware yields different log probability distributions

Legitimate infrastructure change looks like substitution

Randomized Routing at Low Rates Makes Statistical Detection Query-Prohibitive

As p approaches 0, the mixed distribution becomes indistinguishable from the specified model



$$P_{\text{mixed}}(y|x) = (1-p) * P_{\text{spec}}(y|x) + p * P_{\text{sub}}(y|x)$$

As $p \rightarrow 0$, $P_{\text{mixed}} \rightarrow P_{\text{spec}}$

Low p = Hard Detection

Smaller p means fewer substitute samples, making the distribution nearly identical

Exponential Query Cost

Detection requires exponentially more queries as p decreases, becoming cost-prohibitive

Query Budget vs. Substitution Rate



Adversarial advantage: Provider can make detection arbitrarily expensive by choosing a low enough substitution rate p

Software-Only Auditing Methods Are Fundamentally Unreliable

Three independent failure modes make software-only detection insufficient for production use



Text-Based Statistical Tests

METHOD TYPE

MMD, permutation tests
on output text distributions

WEAKNESS

Impractically large query
budgets required; fails
against subtle substitutions

PRACTICAL LIMITATION

FP8 quantization produces
output within error bars of
original model

UNRELIABLE



Log-Probability Methods

METHOD TYPE

Token-level log probability
fingerprinting vs. reference

WEAKNESS

Defeated by production-level
inference nondeterminism
across software and hardware

PRACTICAL LIMITATION

Cannot distinguish legitimate
infra changes from actual
model substitution

UNRELIABLE



Adversarial Countermeasures

METHOD TYPE

Benchmark evasion +
randomized substitution

WEAKNESS

Provider can detect and
route benchmark queries
to genuine model

PRACTICAL LIMITATION

Low-p randomized routing
makes all statistical tests
query-prohibitive

UNRELIABLE

Trusted Execution Environments Provide Provable Cryptographic Model Integrity

TEEs provide hardware-level attestation that the correct model weights are loaded and executed



Provable

Cryptographic proof that specific model weights are loaded and the correct inference code is running



Efficient

Only modest performance overhead compared to untrusted execution; practical for production deployments



Robust

No software-based evasion possible; hardware enclaves prevent tampering even by the infrastructure provider

Property	Software Methods	TEE Attestation
Detection Reliability	Unreliable	Provable
Evasion Resistance	Gameable	Tamper-proof
Query Cost	Prohibitive	Minimal
Infrastructure Dependency	Version-sensitive	Hardware-agnostic
Deployment Status	Research only	Available today

Practical Deployment

NVIDIA Confidential Computing on H100 GPUs provides TEE capabilities for LLM inference

Ready for production use today

vs. Zero-Knowledge Proofs

ZKPs are computationally prohibitive for large model verification

TEEs: practical and deployable

ZKPs: infeasible at scale

KEY TAKEAWAYS

Close the Verification Gap

with Hardware Attestation

1 Model substitution is economically motivated
and practically undetectable via software methods

2 FP8 quantization barely affects benchmarks
attractive substitution but nearly undetectable

3 Inference nondeterminism kills log-prob
fingerprinting across software and hardware

4 Randomized substitution defeats
statistical tests with exponential query cost

5 TEEs are the only currently viable path to verifiable LLM APIs
Hardware attestation provides provable, efficient, and robust model integrity guarantees

Call to Action

The research community and industry should adopt hardware attestation standards
as the foundation for trustworthy LLM API services